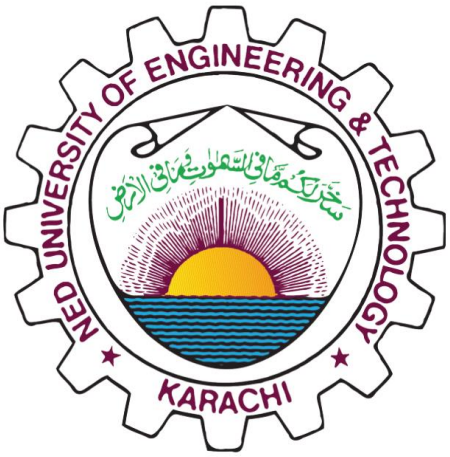


DEPARTMENT OF COMPUTER & INFORMATION SYSTEMS ENGINEERING

Problem Based Learning Report

Course Code: **CS-434**

CourseTitle: **Bioinformatics**

Group Members

Fahad Naeem
CS-23055

Ateeb Hussain
CS-23069

Contents

1. Overview 3
2. Alignment: Local Alignment with Scoring Matrix (LOCA) 3
2.1 Problem statement 3
2.2 Approach 3
2.3 Python code 3
2.4 Explanation of the implementation 6
2.5 Output snapshot 7
3. Alignment: Global Alignment with Affine Gap Penalty (GAFF) 8
3.1 Problem statement 8
3.2 Approach 9
3.3 Python code 9
3.4 Explanation of the implementation 11
3.5 Output snapshot 12
4. String Algorithms: Maximum Matchings and RNA Secondary Structures (MMCH) 14
4.1 Problem statement 14
4.2 Approach 14
4.3 Python code 14
4.4 Explanation of the implementation 15
4.5 Output snapshot 16
5 Conclusion 16

1. Overview

This report fulfills the deliverables for the CS-434 Bioinformatics Problem Based Learning assignment. It contains the problem statements, Python implementations, logic explanations, and output snapshots for three Rosalind problems covering Alignment, Dynamic Programming, and String Algorithms.

2. Alignment: Local Alignment with Scoring Matrix (LOCA)

2.1 Problem statement

A local alignment of two strings s and t is an alignment of substrings r and u of s and t , respectively. Let opt(r,u) denote the score of an optimal alignment of r and u with respect to some predetermined alignment score.

Given: two protein strings s and t in FASTA format.

Return: A maximum alignment score along with substrings r and u of s and t , respectively, which produce this maximum alignment score (multiple solutions may exist, in which case you may output any one). Use The *PAM250* scoring matrix. Linear gap penalty equal to 5.

Sample: s = MEANLYPRTEINSTRING, t = PLEASANTLYEINSTEIN ⇒ **23**, r = LYPRTEINSTRIN, u = LYEINSTEIN.

2.2 Approach

This is the Smith–Waterman algorithm. Let $H(i,j)$ be the best score of a local alignment that ends at s_i and t_j, $S(a,b)$ the PAM250 score and $g = 5$:

$$H(i,0) = H(0,j) = 0,$$

$$H(i,j) = \max \{\ 0,\quad H(i-1,j-1) + S(s_i,t_j),\quad H(i-1,j) - g,\quad H(i,j-1) - g\ \}.$$

The extra 0 lets an alignment restart anywhere, so no cell is ever negative. The answer is the largest value in the table; the traceback starts at that cell and follows the stored moves until a cell with value 0 is reached. The table has $(n+1)(m+1)$ cells and each costs $O(1)$, so time and space are $O(nm)$. The program keeps every move that ties for the best value, so it can list every distinct optimal alignment.

2.3 Python code

```
AA = "ACDEFGHIKLMNPQRSTVWY"
_PAM250 = [
   [ 2,-2, 0, 0,-3, 1,-1,-1,-1,-2,-1, 0, 1, 0,-2, 1, 1, 0,-6,-3],
   [-2,12,-5,-5,-4,-3,-3,-2,-5,-6,-5,-4,-3,-5,-4, 0,-2,-2,-8, 0],
   [ 0,-5, 4, 3,-6, 1, 1,-2, 0,-4,-3, 2,-1, 2,-1, 0, 0,-2,-7,-4],
   [ 0,-5, 3, 4,-5, 0, 1,-2, 0,-3,-2, 1,-1, 2,-1, 0, 0,-2,-7,-4],
   [-3,-4,-6,-5, 9,-5,-2, 1,-5, 2, 0,-3,-5,-5,-4,-3,-3,-1, 0, 7],
   [ 1,-3, 1, 0,-5, 5,-2,-3,-2,-4,-3, 0, 0,-1,-3, 1, 0,-1,-7,-5],
   [-1,-3, 1, 1,-2,-2, 6,-2, 0,-2,-2, 2, 0, 3, 2,-1,-1,-2,-3, 0],
   [-1,-2,-2,-2, 1,-3,-2, 5,-2, 2, 2,-2,-2,-2,-2,-1, 0, 4,-5,-1],
   [-1,-5, 0, 0,-5,-2, 0,-2, 5,-3, 0, 1,-1, 1, 3, 0, 0,-2,-3,-4],
   [-2,-6,-4,-3, 2,-4,-2, 2,-3, 6, 4,-3,-3,-2,-3,-3,-2, 2,-2,-1],
   [-1,-5,-3,-2, 0,-3,-2, 2, 0, 4, 6,-2,-2,-1, 0,-2,-1, 2,-4,-2],
   [ 0,-4, 2, 1,-3, 0, 2,-2, 1,-3,-2, 2, 0, 1, 0, 1, 0,-2,-4,-2],
   [ 1,-3,-1,-1,-5, 0, 0,-2,-1,-3,-2, 0, 6, 0, 0, 1, 0,-1,-6,-5],
   [ 0,-5, 2, 2,-5,-1, 3,-2, 1,-2,-1, 1, 0, 4, 1,-1,-1,-2,-5,-4],
   [-2,-4,-1,-1,-4,-3, 2,-2, 3,-3, 0, 0, 0, 1, 6, 0,-1,-2, 2,-4],
   [ 1, 0, 0, 0,-3, 1,-1,-1, 0,-3,-2, 1, 1,-1, 0, 2, 1,-1,-2,-3],
   [ 1,-2, 0, 0,-3, 0,-1, 0, 0,-2,-1, 0, 0,-1,-1, 1, 3, 0,-5,-3],
   [ 0,-2,-2,-2,-1,-1,-2, 4,-2, 2, 2,-2,-1,-2,-2,-1, 0, 4,-6,-2],
   [-6,-8,-7,-7, 0,-7,-3,-5,-3,-2,-4,-4,-6,-5, 2,-2,-5,-6,17, 0],
```

```
    [-3, 0,-4,-4, 7,-5, 0,-1,-4,-1,-2,-2,-5,-4,-4,-3,-3,-2, 0,10],
]
PAM = {a: {b: _PAM250[i][j] for j, b in enumerate(AA)} for i, a in enumerate(AA)}
ARROW = {(1, 1): "\\", (1, 0): "^", (0, 1): "<"}      # diagonal, up, left

def clean_sequence(raw):
    """Strip FASTA header lines and spaces; validate the 20 amino-acid letters."""
    lines = [l.strip() for l in raw.strip().splitlines() if not l.strip().startswith(">")]
    seq = "".join("".join(lines).split()).upper()
    if not seq:
        raise ValueError("Sequence cannot be empty.")
    bad = sorted(set(seq) - set(AA))
    if bad:
        raise ValueError(f"Invalid amino acid symbol(s): {bad}")
    return seq

def smith_waterman(s, t, gap):
    """Fill H, keep every tied direction, then trace back from every peak cell."""
    n, m = len(s), len(t)
    H = [[0] * (m + 1) for _ in range(n + 1)]
    T = [[[] for _ in range(m + 1)] for _ in range(n + 1)]
    best, peaks = 0, []
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            moves = {(1, 1): H[i-1][j-1] + PAM[s[i-1]][t[j-1]],
                     (1, 0): H[i-1][j] - gap, (0, 1): H[i][j-1] - gap}
            H[i][j] = v = max(0, *moves.values())
            if v > 0:
                T[i][j] = [mv for mv, sc in moves.items() if sc == v]
            if v > best:
                best, peaks = v, [(i, j)]
            elif v == best and v > 0:
                peaks.append((i, j))

    found, seen = [], set()

    def trace(i, j, a, b, path, end):
        if H[i][j] == 0:                    # reached a zero cell: alignment complete
            key = (i + 1, j + 1, end, a, b)
            if key not in seen:
                seen.add(key)
                found.append({"start": (i + 1, j + 1), "end": end, "a": a, "b": b, "path": set(path)})
            return
        for di, dj in T[i][j]:
            trace(i - di, j - dj, (s[i-1] if di else "-") + a,
                  (t[j-1] if dj else "-") + b, path + [(i - di, j - dj)], end)

    for pi, pj in peaks:
        trace(pi, pj, "", "", [(pi, pj)], (pi, pj))
    return {"H": H, "T": T, "score": best, "all": found,
            "path": set().union(*(f["path"] for f in found)) if found else set()}

def show_matrix(s, t, r):
    """Print H[i][j]; cells on a traceback path are shown as [value]."""
    n, m, H, T = len(s), len(t), r["H"], r["T"]
    ei, ej = r["all"][0]["end"] if r["all"] else (n, m)
    i0, i1 = (0, n) if n <= 22 else (max(0, ei - 14), ei)
    j0, j1 = (0, m) if m <= 22 else (max(0, ej - 14), ej)
```

```
    print("      " + "".join(f"{('-' if j == 0 else t[j-1]) + f'({j})':^7}" for j in range(j0, j1 + 1)))
    for i in range(i0, i1 + 1):
        row = f"{('-' if i == 0 else s[i-1]) + f'({i})':<6}"
        for j in range(j0, j1 + 1):
            cell = f"{H[i][j]}{''.join(ARROW[d] for d in T[i][j])}"
            row += f"{'[' + cell + ']' if (i, j) in r['path'] else cell:^7}"
        print(row)
    print("  [value] = traceback path;  \\ diagonal  ^ up (gap in t)  < left (gap in s)")

def run_case(s, t, gap):
    r = smith_waterman(s, t, gap)
    first = r["all"][0] if r["all"] else {"a": "", "b": "", "start": (1, 1), "end": (0, 0)}
    a, b = first["a"], first["b"]
    mid = "".join(" " if "-" in (x, y) else "|" if x == y else ":" if PAM[x][y] > 0
                  else "." if PAM[x][y] == 0 else " " for x, y in zip(a, b))
    ident, pos = mid.count("|"), mid.count("|") + mid.count(":")
    L = len(a) or 1

    print(f"\n[STEP 1] INPUT\n  s = {s}  ({len(s)} aa)\n  t = {t}  ({len(t)} aa)")
    print(f"  Matrix = PAM250, linear gap penalty = {gap}")
    print(f"\n[STEP 2] SMITH-WATERMAN MATRIX H (peak score {r['score']})")
    show_matrix(s, t, r)
    print(f"\n[STEP 3] OPTIMAL LOCAL ALIGNMENT  (s[{first['start'][0]}..{first['end'][0]}], "
          f"t[{first['start'][1]}..{first['end'][1]}])")
    print(f"  s : {a}\n      {mid}\n  t : {b}")
    print(f"  Identity {ident}/{L} ({ident / L:.1%}), similarity {pos}/{L} ({pos / L:.1%}), "
          f"gaps {a.count('-') + b.count('-')}/{L}")
    print("\n[STEP 4] FINAL ROSALIND 'LOCA' OUTPUT")
    print(r["score"], a.replace("-", ""), b.replace("-", ""), sep="\n")
    print(f"\n[STEP 5] ALL DISTINCT OPTIMAL ALIGNMENTS: {len(r['all'])}")
    for k, f in enumerate(r["all"], 1):
        print(f"  #{k}  s[{f['start'][0]}..{f['end'][0]}] t[{f['start'][1]}..{f['end'][1]}]\n"
              f"      {f['a']}\n      {f['b']}")

def read_input():
    """Read two validated sequences and a gap penalty (default 5)."""
    seqs = []
    for k in (1, 2):
        while True:
            try:
                seqs.append(clean_sequence(input(f"  Enter Sequence {k}: ")))
                break
            except ValueError as e:
                print("  Invalid:", e)
    while True:
        raw = input("  Enter gap penalty [5]: ").strip() or "5"
        if raw.isdigit():
            return seqs[0], seqs[1], int(raw)
        print("  Please enter a whole number.")

def main():
    sample = ("MEANLYPRTEINSTRING", "PLEASANTLYEINSTEIN", 5)
    print("\n=== LOCAL ALIGNMENT (ROSALIND: LOCA) ===")
    print("[1] Sample Case  [2] Custom Case  [3] Sample + Custom  [4] Exit")
    choice = input("Select an option [1-4]: ").strip()
    if choice in ("1", "3"):
        print("\nRunning the sample case...")
        run_case(*sample)
```

```
    if choice in ("2", "3"):
        run_case(*read_input())
    if choice not in ("1", "2", "3", "4"):
        print("Invalid choice.")

if __name__ == "__main__":
    try:
        main()
    except (KeyboardInterrupt, EOFError):
        print("\nProgram stopped by user.")
```

2.4 Explanation of the implementation

1. **Scoring matrix.** `_PAM250` is a 20 × 20 list of scores in the order of `AA`; the dictionary `PAM` makes lookups such as `PAM['W']['W']` = 17 easy.
2. **Input check.** `clean_sequence()` removes FASTA header lines and spaces, converts to upper case and rejects an empty string or any letter that is not one of the 20 amino acids.
3. **Table.** `smith_waterman()` fills `H` with zeros on the borders and, for every cell, takes the best of 0, diagonal, up and left. `T` remembers every move that ties for that best positive value, and `peaks` collects all cells that reach the highest score.
4. **Traceback.** The recursive function `trace()` starts at each peak and follows the stored moves until it reaches a cell with value 0. The aligned strings are built by adding letters (or –) to the front, and the set `seen` removes duplicates, so the result is every distinct optimal alignment.
5. **Matrix display.** `show_matrix()` prints `H` with the move arrows and writes cells on a traceback path as `[value]`. If a sequence is longer than 22 letters, only a 15 × 15 window ending at the peak is printed.
6. **Report.** `run_case()` prints the inputs, the matrix, the first optimal alignment with a middle line (`|` identical, `:` positive PAM250 score, `.` zero), identity, similarity and gap counts, the final Rosalind output (score, *r*, *u*) and the list of all distinct optimal alignments.
7. **Menu.** `read_input()` re-asks until both sequences are valid and the gap penalty is a whole number (default 5). `main()` offers sample, custom, sample plus custom, and exit.

2.5 Output snapshot

Run `LOCA.py`, choose option 1 for the sample case and option 2 for a custom case. The screen shows the matrix *H*, the optimal alignment, and the final Rosalind output (the score followed by the two substrings).

```
=== LOCAL ALIGNMENT (ROSALIND: LOCA) ===
[1] Sample Case  [2] Custom Case  [3] Sample + Custom  [4] Exit
Select an option [1-4]: 1

Running the sample case...

[STEP 1] INPUT
  s = MEANLYPRTEINSTRING  (18 aa)
  t = PLEASANTLYEINSTEIN  (18 aa)
  Matrix = PAM250, linear gap penalty = 5

[STEP 2] SMITH-WATERMAN MATRIX H (peak score 23)
      -(0)   P(1)   L(2)   E(3)   A(4)   S(5)   A(6)   N(7)   T(8)   L(9)   Y(10)  E(11)  I(12)  N(13)  S(14)  T(15)  E(16)  I(17)  N(18)
-(0)     0    [0]     0      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0      0
M(1)     0      0   [4\]     0      0      0      0      0      0     4\      0      0     2\      0      0      0      0     2\      0
E(2)     0      0      0   [8\]    3<      0      0     1\      0      0      0     4\      0     3\      0      0     4\      0     3\
A(3)     0     1\      0     3^   [10\]   5<     2\      0     2\      0      0      0     3\      0     4\     1\      0     3\      0
N(4)     0      0      0     1\     5^   [11\]   6<     4\    [0]       0      0      0     1\      0     5\     1\     4\     2\      0     5\
L(5)     0      0     6\     1<      0     6^    [9\]   4<     2\    [6\]    1<      0     3\      0     2\      0     1\     4\      0
Y(6)     0      0     1^     2\      0     1^     4^   [7\]    2<     1\^  [16\]   11<    6<     1\<     0      0      0      0     2\
P(7)     0     6\     1<      0     3\     1\     2\     4\   [7\]    2<   [11^]   15\    10<     6\     2\      0      0      0      0
R(8)     0     1^     3\      0      0     3\      0     2\     3\   [4\]   [6^]   10\^   13\    10\     6\     1\<     0      0      0
T(9)     0      0      0     3\     1\     1\     4\      0     5\     1\   [1\^]   6\    10\    13\    11\     9\     4<      0      0
E(10)    0      0      0     4\     3\     1\     1\     5\      0     2\      0    [5\]    5^    11\    13\    11\    13\     8<     3<
I(11)    0      0     2\      0     3\     2\      0      0     5\     2\     1\      0   [10\]   6^     10\    13\     9\     18\    13<
N(12)    0      0      0     3\      0     4\     2\     2\      0     2\      0     2\    5^    [12\]   7\<    10\    14\    13^    20\
S(13)    0     1\      0      0     4\     2\     5\     3\     3\      0      0      0     1\     7^    [14\]   9<     10\    13\    15^
T(14)    0      0      0      0     1\     5\     3\     5\     6\    1\<     0      0      0     2^     9^    [17\]   12<    10\    13\
R(15)    0      0      0      0      0     1\     3\     3\     4\     3\      0      0      0      0     4^     12^   [16\]   11<    10\
I(16)    0      0     2\      0      0      0      0     1\     3\     6\     2\      0     5\      0      0     7^     11^   [21\]   16<
N(17)    0      0      0     3\      0     1\      0     2\     1\     1^     4\     3\      0     7\     2<     2^     8\     16^   [23\]
G(18)    0      0      0      0     4\     1\     2\      0     2\      0      0     4\      0     2^     8\     3<     3^     11^    18^
  [value] = traceback path;  \ diagonal  ^ up (gap in t)  < left (gap in s)

[STEP 3] OPTIMAL LOCAL ALIGNMENT  (s[1..17], t[2..18])
  s : MEANLYPRTEINSTRIN
      :||:  .  ||||| ||
  t : LEASANTLYEINSTEIN
  Identity 9/17 (52.9%), similarity 11/17 (64.7%), gaps 0/17

[STEP 4] FINAL ROSALIND 'LOCA' OUTPUT
23
MEANLYPRTEINSTRIN
LEASANTLYEINSTEIN

[STEP 5] ALL DISTINCT OPTIMAL ALIGNMENTS: 2
  #1  s[1..17] t[2..18]
      MEANLYPRTEINSTRIN
      LEASANTLYEINSTEIN
  #2  s[5..17] t[9..18]
      LYPRTEINSTRIN
      LY---EINSTEIN
```

LOCA.py, sample case.

```
=== LOCAL ALIGNMENT (ROSALIND: LOCA) ===
[1] Sample Case  [2] Custom Case  [3] Sample + Custom  [4] Exit
Select an option [1-4]: 2
  Enter Sequence 1: AWILDTYPEAA
  Enter Sequence 2: EWILDVTYP
  Enter gap penalty [5]: 5

[STEP 1] INPUT
  s = AWILDTYPEAA  (11 aa)
  t = EWILDVTYP  (9 aa)
  Matrix = PAM250, linear gap penalty = 5

[STEP 2] SMITH-WATERMAN MATRIX H (peak score 46)
       -(0)   E(1)   W(2)   I(3)   L(4)   D(5)   V(6)   T(7)   Y(8)   P(9)
-(0)      0      0      0      0      0      0      0      0      0      0
A(1)      0    [0]      0      0      0      0      0     1\      0     1\
W(2)      0      0   [17\]   12<     7<     2<      0      0     1\      0
I(3)      0      0    12^   [22\]   17<    12<     7<     2<      0      0
L(4)      0      0     7^    17^   [28\]   23<    18<    13<     8<     3<
D(5)      0     3\     2^    12^    23^   [32\]  [27<]   22<    17<    12<
T(6)      0      0      0     7^    18^    27^    32\   [30\]   25<    20<
Y(7)      0      0      0     2^    13^    22^    27^    29\   [40\]   35<
P(8)      0      0      0      0     8^    17^    22^    27\    35^   [46\]
E(9)      0     4\      0      0     3^    12^    17^   22\^    30^    41^
A(10)     0      0      0      0      0     7^   12\^    18\    25^    36^
A(11)     0      0      0      0      0     2^     7\^  13\^    20^    31^
  [value] = traceback path;  \ diagonal  ^ up (gap in t)  < left (gap in s)

[STEP 3] OPTIMAL LOCAL ALIGNMENT  (s[2..8], t[2..9])
  s : WILD-TYP
      |||| |||
  t : WILDVTYP
  Identity 7/8 (87.5%), similarity 7/8 (87.5%), gaps 1/8

[STEP 4] FINAL ROSALIND 'LOCA' OUTPUT
46
WILDTYP
WILDVTYP

[STEP 5] ALL DISTINCT OPTIMAL ALIGNMENTS: 1
  #1  s[2..8] t[2..9]
      WILD-TYP
      WILDVTYP
```

LOCA.py, custom case.

3. Alignment: Global Alignment with Affine Gap Penalty (GAFF)

3.1 Problem statement

An *affine gap penalty* is written as $a+b\cdot(L-1)$, where L is the length of the gap, a is a positive constant called the *gap opening penalty*, and b is a positive constant called the *gap extension penalty*. We can view the *gap opening penalty* as charging for the first gap symbol, and the *gap extension penalty* as charging for each subsequent symbol added to the gap.

Given: two protein strings *s* and *t* in FASTA format.

Return: The maximum alignment score between *s* and *t* , followed by two augmented strings s' and t' representing an optimal alignment of *s* and *t* . Use: The *BLOSUM62* scoring matrix. *Gap opening penalty* equal to 11. *Gap extension penalty* equal to 1.

Sample: *s* = PRTEINS, *t* = PRTWPSEIN ⇒ **8**, with PRT---EINS and PRTWPSEIN-.

3.2 Approach

The cost of a gap depends on whether it is just starting or continuing, so one table is not enough. The algorithm (Gotoh) uses three tables, each ending in a different state: $M(i,j)$ ends with s_i paired to t_j; $X(i,j)$ ends with s_i against a gap; $Y(i,j)$ ends with a gap against t_j. With S the BLOSUM62 score:

$$M(i,j) = S(s_i,t_j) + \max \{ M(i-1,j-1), X(i-1,j-1), Y(i-1,j-1) \},$$
$$X(i,j) = \max \{ M(i-1,j) - \sigma, \quad X(i-1,j) - \varepsilon \},$$
$$Y(i,j) = \max \{ M(i,j-1) - \sigma, \quad Y(i,j-1) - \varepsilon \}.$$

The start is $M(0,0) = 0$, $X(i,0) = -\sigma - (i-1)\varepsilon$, $Y(0,j) = -\sigma - (j-1)\varepsilon$, and every other border cell is $-\infty$. The answer is the largest of $M(n,m)$, $X(n,m)$ and $Y(n,m)$. There are three tables of $(n+1)(m+1)$ cells and each cell costs $O(1)$, so time and space are $O(nm)$.

3.3 Python code

```
AA = "ARNDCQEGHILKMFPSTWYV"
_BLOSUM = [
   [ 4,-1,-2,-2, 0,-1,-1, 0,-2,-1,-1,-1,-1,-2,-1, 1, 0,-3,-2, 0],
   [-1, 5, 0,-2,-3, 1, 0,-2, 0,-3,-2, 2,-1,-3,-2,-1,-1,-3,-2,-3],
   [-2, 0, 6, 1,-3, 0, 0, 0, 1,-3,-3, 0,-2,-3,-2, 1, 0,-4,-2,-3],
   [-2,-2, 1, 6,-3, 0, 2,-1,-1,-3,-4,-1,-3,-3,-1, 0,-1,-4,-3,-3],
   [ 0,-3,-3,-3, 9,-3,-4,-3,-3,-1,-1,-3,-1,-2,-3,-1,-1,-2,-2,-1],
   [-1, 1, 0, 0,-3, 5, 2,-2, 0,-3,-2, 1, 0,-3,-1, 0,-1,-2,-1,-2],
   [-1, 0, 0, 2,-4, 2, 5,-2, 0,-3,-3, 1,-2,-3,-1, 0,-1,-3,-2,-2],
   [ 0,-2, 0,-1,-3,-2,-2, 6,-2,-4,-4,-2,-3,-3,-2, 0,-2,-2,-3,-3],
   [-2, 0, 1,-1,-3, 0, 0,-2, 8,-3,-3,-1,-2,-1,-2,-1,-2,-2, 2,-3],
   [-1,-3,-3,-3,-1,-3,-3,-4,-3, 4, 2,-3, 1, 0,-3,-2,-1,-3,-1, 3],
   [-1,-2,-3,-4,-1,-2,-3,-4,-3, 2, 4,-2, 2, 0,-3,-2,-1,-2,-1, 1],
   [-1, 2, 0,-1,-3, 1, 1,-2,-1,-3,-2, 5,-1,-3,-1, 0,-1,-3,-2,-2],
   [-1,-1,-2,-3,-1, 0,-2,-3,-2, 1, 2,-1, 5, 0,-2,-1,-1,-1, 1,-1],
   [-2,-3,-3,-3,-2,-3,-3,-3,-1, 0, 0,-3, 0, 6,-4,-2,-2, 1, 3,-1],
   [-1,-2,-2,-1,-3,-1,-1,-2,-2,-3,-3,-1,-2,-4, 7,-1,-1,-4,-3,-2],
   [ 1,-1, 1, 0,-1, 0, 0, 0,-1,-2,-2, 0,-1,-2,-1, 4, 1,-3,-2,-2],
   [ 0,-1, 0,-1,-1,-1,-1,-2,-2,-1,-1,-1,-1,-2,-1, 1, 5,-2,-2, 0],
   [-3,-3,-4,-4,-2,-2,-3,-2,-2,-3,-2,-3,-1, 1,-4,-3,-2,11, 2,-3],
   [-2,-2,-2,-3,-2,-1,-2,-3, 2,-1,-1,-2,-1, 3,-3,-2,-2, 2, 7,-1],
   [ 0,-3,-3,-3,-1,-2,-2,-3,-3, 3, 1,-2, 1,-1,-2,-2, 0,-3,-1, 4],
]
B = {a: {b: _BLOSUM[i][j] for j, b in enumerate(AA)} for i, a in enumerate(AA)}
NEG = -10**12
MOVES = [(1, 1), (1, 0), (0, 1)]          # state 0 = M (pair), 1 = X (gap in t), 2 = Y (gap in s)

def clean_sequence(raw):
   """Strip FASTA header lines and spaces; validate the 20 amino-acid letters."""
   lines = [l.strip() for l in raw.strip().splitlines() if l.strip() and not l.strip().startswith(">")]
   seq = "".join("".join(lines).split()).upper()
   if not seq:
      raise ValueError("Sequence cannot be empty.")
   bad = sorted(set(seq) - set(AA))
   if bad:
      raise ValueError(f"Invalid amino acid symbols: {bad}")
   return seq

def affine_global_alignment(s, t, go=11, ge=1):
   """Three-state DP (M, X, Y) with predecessor pointers, then traceback from (n, m)."""
   n, m = len(s), len(t)
   grid = lambda v: [[v] * (m + 1) for _ in range(n + 1)]
   M, X, Y = grid(NEG), grid(NEG), grid(NEG)
   P = [grid(None), grid(None), grid(None)]          # predecessor state for M, X, Y
   M[0][0] = 0
   for i in range(1, n + 1):
      X[i][0], P[1][i][0] = -go - (i - 1) * ge, (0 if i == 1 else 1)
   for j in range(1, m + 1):
      Y[0][j], P[2][0][j] = -go - (j - 1) * ge, (0 if j == 1 else 2)

   best = lambda open_, ext, k: (open_, 0) if open_ >= ext else (ext, k)
   for i in range(1, n + 1):
      for j in range(1, m + 1):
```

```
            prev = (M[i-1][j-1], X[i-1][j-1], Y[i-1][j-1])
            k = max(range(3), key=prev.__getitem__)
            M[i][j], P[0][i][j] = prev[k] + B[s[i-1]][t[j-1]], k
            X[i][j], P[1][i][j] = best(M[i-1][j] - go, X[i-1][j] - ge, 1)
            Y[i][j], P[2][i][j] = best(M[i][j-1] - go, Y[i][j-1] - ge, 2)

    end = (M[n][m], X[n][m], Y[n][m])
    k = max(range(3), key=end.__getitem__)
    score, i, j, a, b, path = end[k], n, m, "", "", set()
    while i or j:
        path.add((i, j, k))
        di, dj = MOVES[k]
        a, b = (s[i-1] if di else "-") + a, (t[j-1] if dj else "-") + b
        k, i, j = P[k][i][j], i - di, j - dj
    path.add((0, 0, k))
    return {"score": score, "a": a, "b": b, "M": M, "X": X, "Y": Y, "P": P, "path": path}

def rescore(a, b, go, ge):
    """Independent check: add BLOSUM62 pairs and charge go + ge*(L-1) per gap run."""
    total, in_gap = 0, False
    for x, y in zip(a, b):
        if "-" in (x, y):
            total -= ge if in_gap else go
            in_gap = True
        else:
            total += B[x][y]
            in_gap = False
    return total

def show_matrices(s, t, r):
    if len(s) > 22 or len(t) > 22:
        print("  DP matrices omitted (a sequence is longer than 22 aa).")
        return
    w = 7
    cols = ["-(0)"] + [f"{t[j-1]}({j})" for j in range(1, len(t) + 1)]
    head = "i\\j".center(w) + "".join(f"{c:^{w}}" for c in cols)
    for k, (name, what, arrow) in enumerate((("M", "paired residues", "\\"),
                                             ("X", "gap in t", "^"), ("Y", "gap in s", "<"))):
        print(f"\n  State {name}: {what}\n{head}")
        for i in range(len(s) + 1):
            row = f"{('-' if i == 0 else s[i-1]) + f'({i})':^{w}}"
            for j in range(len(t) + 1):
                v = r[name][i][j]
                cell = "-INF" if v < NEG // 2 else str(v) + ("" if r["P"][k][i][j] is None else arrow)
                if v > NEG // 2 and (i, j, k) in r["path"]:
                    cell = f"[{cell}]"
                row += f"{cell:^{w}}"
            print(row)
    print("\n  [cell] = traceback path;  -INF = unreachable;  \\ pair  ^ gap in t  < gap in s")

def run_case(s, t, go=11, ge=1):
    r = affine_global_alignment(s, t, go, ge)
    a, b = r["a"], r["b"]
    pairs = [(x, y) for x, y in zip(a, b) if "-" not in (x, y)]
    ident = sum(x == y for x, y in pairs)
    mid = "".join(" " if "-" in (x, y) else "|" if x == y else ":" if B[x][y] > 0
                  else "." if B[x][y] == 0 else " " for x, y in zip(a, b))

    print(f"\n[STEP 1] INPUT\n  s = {s}  ({len(s)} aa)\n  t = {t}  ({len(t)} aa)")
    print(f"  Matrix = BLOSUM62, gap opening = {go}, gap extension = {ge}")
    print(f"  A gap of length L costs {go} + {ge} x (L - 1)")
    print(f"\n[STEP 2] AFFINE-GAP DP MATRICES (optimal score {r['score']})")
    show_matrices(s, t, r)
    print(f"\n[STEP 3] OPTIMAL GLOBAL ALIGNMENT ({len(a)} columns)")
    print(f"  s : {a}\n      {mid}\n  t : {b}")
    print(f"  Identity {ident}/{len(pairs)} paired residues, gap symbols {len(a) - len(pairs)}")
    print("\n[STEP 4] FINAL ROSALIND 'GAFF' OUTPUT")
    print(r["score"], a, b, sep="\n")
    check = rescore(a, b, go, ge)
    print("\n[STEP 5] VALIDATION")
```

```
    print(f"  Recalculated score = {check} -> {'MATCH' if check == r['score'] else 'MISMATCH'}")
    print(f"  Gaps removed give back s and t: {a.replace('-', '') == s and b.replace('-', '') == t}")

def read_input():
    seqs = []
    for k in (1, 2):
        while True:
            try:
                seqs.append(clean_sequence(input(f"  Enter Sequence {k}: ")))
                break
            except ValueError as e:
                print("  Invalid:", e)
    return seqs

def main():
    sample = ("PRTEINS", "PRTWPSEIN")
    print("\n=== GLOBAL ALIGNMENT, AFFINE GAP (ROSALIND: GAFF) ===")
    print("[1] Sample Case  [2] Custom Case  [3] Sample + Custom  [4] Exit")
    choice = input("Select an option [1-4]: ").strip()
    if choice in ("1", "3"):
        print("\nRunning the sample case...")
        run_case(*sample)
    if choice in ("2", "3"):
        run_case(*read_input())
    if choice not in ("1", "2", "3", "4"):
        print("Invalid choice.")

if __name__ == "__main__":
    try:
        main()
    except (KeyboardInterrupt, EOFError):
        print("\nProgram stopped by user.")
```

3.4 Explanation of the implementation

1. **Scoring matrix.** `_BLOSUM` is the 20 × 20 BLOSUM62 table in the order of `AA`; the dictionary `B` is used as `B['W']['W']` = 11.
2. **Input check.** `clean_sequence()` strips FASTA headers and spaces, upper-cases the text and rejects an empty string or a letter outside the 20 amino acids.
3. **Tables.** `affine_global_alignment()` creates `M`, `X` and `Y`, using `NEG` = -10^{12} as $-\infty$, and a pointer table `P` for each state that records which state the value came from. The borders follow the start conditions above.
4. **Recurrence.** `M` takes the best of the three diagonal predecessors. `X` and `Y` compare opening a gap from `M` against extending an existing gap; on a tie the opening is chosen, and for `M` ties prefer the `M` state.
5. **Traceback.** It starts at (n,m) in the state with the highest value. The list `MOVES` gives the step for each state (diagonal, up, left), letters or `-` are added to the front, and `P` gives the previous state. Cells visited are stored in `path`.
6. **Check.** `rescore()` recomputes the score from the two gapped strings alone: the BLOSUM62 value for each pair column, σ for the first symbol of a gap run and ε for each further one.
7. **Display.** `show_matrices()` prints `M`, `X` and `Y` with arrows, marks the traceback cells as `[value]` and shows unreachable cells as `-INF`. It is skipped when a sequence is longer than 22 letters.
8. **Report and menu.** `run_case()` prints the inputs, the matrices, the alignment with a middle line (`|` identical, `:` positive, `.` zero), the final Rosalind output (score and the two gapped strings) and a validation step. `main()` shows the sample / custom / both / exit menu.

3.5 Output snapshot

```
=== GLOBAL ALIGNMENT, AFFINE GAP (ROSALIND: GAFF) ===
[1] Sample Case  [2] Custom Case  [3] Sample + Custom  [4] Exit
Select an option [1-4]: 1

Running the sample case...

[STEP 1] INPUT
  s = PRTEINS  (7 aa)
  t = PRTWPSEIN  (9 aa)
  Matrix = BLOSUM62, gap opening = 11, gap extension = 1
  A gap of length L costs 11 + 1 x (L - 1)

[STEP 2] AFFINE-GAP DP MATRICES (optimal score 8)

  State M: paired residues
  i\j   -(0)   P(1)   R(2)   T(3)   W(4)   P(5)   S(6)   E(7)   I(8)   N(9)
 -(0)    [0]   -INF   -INF   -INF   -INF   -INF   -INF   -INF   -INF   -INF
 P(1)   -INF   [7\]   -13\   -13\   -17\    -7\   -16\   -17\   -20\   -20\
 R(2)   -INF   -13\   [12\]   -5\    -8\    -8\    -8\    -8\   -12\   -10\
 T(3)   -INF   -13\    -5\   [17\]   -1\    -1\     0\    -3\    -4\    -4\
 E(4)   -INF   -14\    -5\     0\     14\    5\     5\    [9\]    0\     2\
 I(5)   -INF   -17\    -9\    -1\     3\     11\    3\     2\    [13\]   -3\
 N(6)   -INF   -17\    -7\    -1\     1\     1\     12\    3\     -1\   [19\]
 S(7)   -INF   -17\    -9\    -1\     1\     1\     5\     12\    1\     3\

  State X: gap in t
  i\j   -(0)   P(1)   R(2)   T(3)   W(4)   P(5)   S(6)   E(7)   I(8)   N(9)
 -(0)   -INF   -INF   -INF   -INF   -INF   -INF   -INF   -INF   -INF   -INF
 P(1)   -11^   -INF   -INF   -INF   -INF   -INF   -INF   -INF   -INF   -INF
 R(2)   -12^    -4^   -24^   -24^   -28^   -18^   -27^   -28^   -31^   -31^
 T(3)   -13^    -5^    1^    -16^   -19^   -19^   -19^   -19^   -23^   -21^
 E(4)   -14^    -6^    0^     6^    -12^   -12^   -11^   -14^   -15^   -15^
 I(5)   -15^    -7^    -1^    5^     3^     -6^    -6^    -2^   -11^    -9^
 N(6)   -16^    -8^    -2^    4^     2^     0^     -7^    -3^    2^     -10^
 S(7)   -17^    -9^    -3^    3^     1^     -1^    1^     -4^    1^     [8^]

  State Y: gap in s
  i\j   -(0)   P(1)   R(2)   T(3)   W(4)   P(5)   S(6)   E(7)   I(8)   N(9)
 -(0)   -INF   -11<   -12<   -13<   -14<   -15<   -16<   -17<   -18<   -19<
 P(1)   -INF   -INF    -4<    -5<    -6<    -7<    -8<    -9<   -10<   -11<
 R(2)   -INF   -INF   -24<     1<     0<     -1<    -2<    -3<    -4<    -5<
 T(3)   -INF   -INF   -24<   -16<   [6<]   [5<]   [4<]    3<     2<     1<
 E(4)   -INF   -INF   -25<   -16<   -11<    3<     2<     1<     0<     -1<
 I(5)   -INF   -INF   -28<   -20<   -12<    -8<    0<     -1<    -2<     2<
 N(6)   -INF   -INF   -28<   -18<   -12<   -10<   -10<    1<     0<     -1<
 S(7)   -INF   -INF   -28<   -20<   -12<   -10<   -10<    -6<    1<     0<

  [cell] = traceback path;  -INF = unreachable;  \ pair  ^ gap in t  < gap in s
```

GAFF.py, sample case.

```
=== GLOBAL ALIGNMENT, AFFINE GAP (ROSALIND: GAFF) ===
[1] Sample Case  [2] Custom Case  [3] Sample + Custom  [4] Exit
Select an option [1-4]: 2
  Enter Sequence 1: ALIGNMENT
  Enter Sequence 2: ALIMENT

[STEP 1] INPUT
  s = ALIGNMENT  (9 aa)
  t = ALIMENT  (7 aa)
  Matrix = BLOSUM62, gap opening = 11, gap extension = 1
  A gap of length L costs 11 + 1 x (L - 1)

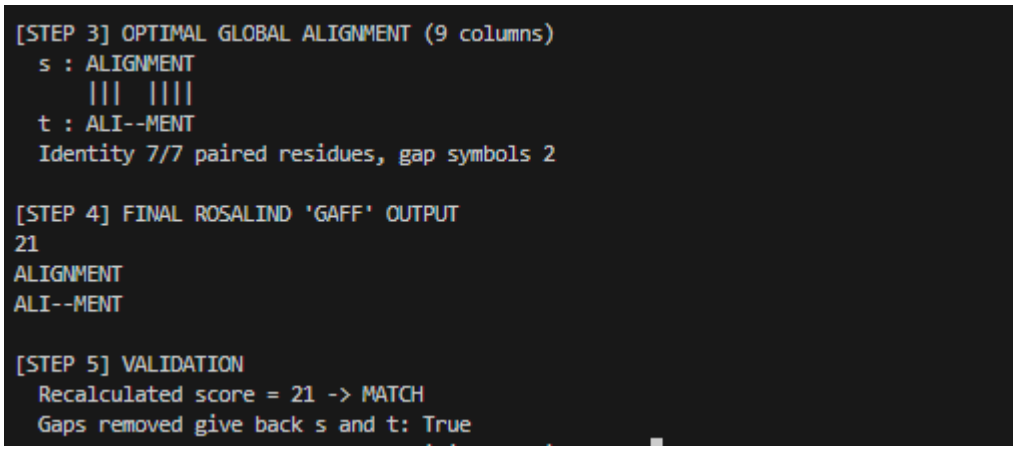
[STEP 2] AFFINE-GAP DP MATRICES (optimal score 21)

  State M: paired residues
  i\j   -(0)   A(1)   L(2)   I(3)   M(4)   E(5)   N(6)   T(7)
 -(0)    [0]   -INF   -INF   -INF   -INF   -INF   -INF   -INF
 A(1)   -INF   [4\]   -12\   -13\   -14\   -15\   -17\   -16\
 L(2)   -INF   -12\   [8\]    -5\    -6\   -12\   -13\   -12\
 I(3)   -INF   -13\    -5\   [12\]   -2\    -7\    -8\    -7\
 G(4)   -INF   -13\   -12\    -7\     9\     -1\     0\     -3\
 N(5)   -INF   -16\   -12\    -7\    -1\     9\     5\     0\
 M(6)   -INF   -16\    -8\    -4\   [5\]    -3\     7\     4\
 E(7)   -INF   -17\   -14\    -9\    -3\   [10\]   -2\     6\
 N(8)   -INF   -19\   -15\   -10\    -4\    -3\   [16\]   -1\
 T(9)   -INF   -18\   -14\    -9\    -4\    -5\    -1\   [21\]

  State X: gap in t
  i\j   -(0)   A(1)   L(2)   I(3)   M(4)   E(5)   N(6)   T(7)
 -(0)   -INF   -INF   -INF   -INF   -INF   -INF   -INF   -INF
 A(1)   -11^   -INF   -INF   -INF   -INF   -INF   -INF   -INF
 L(2)   -12^    -7^   -23^   -24^   -25^   -26^   -28^   -27^
 I(3)   -13^    -8^    -3^   -16^   -17^   -23^   -24^   -23^
 G(4)   -14^    -9^    -4^   [1^]   -13^   -18^   -19^   -18^
 N(5)   -15^   -10^    -5^   [0^]    -2^   -12^   -11^   -14^
 M(6)   -16^   -11^    -6^    -1^    -3^    -2^    -6^   -11^
 E(7)   -17^   -12^    -7^    -2^    -4^    -3^    -4^    -7^
 N(8)   -18^   -13^    -8^    -3^    -5^    -1^    -5^    -5^
 T(9)   -19^   -14^    -9^    -4^    -6^    -2^     5^    -6^

  State Y: gap in s
  i\j   -(0)   A(1)   L(2)   I(3)   M(4)   E(5)   N(6)   T(7)
 -(0)   -INF   -11<   -12<   -13<   -14<   -15<   -16<   -17<
 A(1)   -INF   -INF    -7<    -8<    -9<   -10<   -11<   -12<
 L(2)   -INF   -INF   -23<    -3<    -4<    -5<    -6<    -7<
 I(3)   -INF   -INF   -24<   -16<     1<     0<    -1<    -2<
 G(4)   -INF   -INF   -24<   -23<   -18<    -2<    -3<    -4<
 N(5)   -INF   -INF   -27<   -23<   -18<   -12<    -2<    -3<
 M(6)   -INF   -INF   -27<   -19<   -15<    -6<    -7<    -4<
 E(7)   -INF   -INF   -28<   -25<   -20<   -14<    -1<    -2<
 N(8)   -INF   -INF   -30<   -26<   -21<   -15<   -14<     5<
 T(9)   -INF   -INF   -29<   -25<   -20<   -15<   -16<   -12<

  [cell] = traceback path;  -INF = unreachable;  \ pair  ^ gap in t  < gap in s
```



GAFF.py, custom case.

4. String Algorithms: Maximum Matchings and RNA Secondary Structures (MMCH)

4.1 Problem statement

If we have an RNA string *s* that does not have the same number of occurrences of *'C'* as *'G'* and the same number of occurrences of *'A'* as *'U'*, then the bonding graph of *s* cannot possibly possess a perfect matching among its basepair edges. For example, in fact, most bonding graphs will not contain a perfect matching. In light of this fact, we define a maximum matching in a graph as a matching containing as many edges as possible. A maximum matching of basepair edges will correspond to a way of forming as many base pairs as possible in an RNA string.

Given: An RNA string s of length at most 100.

Return: The total possible number of maximum matchings of basepair edges in the bonding graph of *s*.

Sample: *s* = AUGCUUC ⇒ **6**.

4.2 Approach

The A–U edges and the C–G edges never share a node, so the two groups are counted separately and the results are multiplied. Take the A–U group with a adenines and u uracils, and let $k = \min(a,u)$ and $K = \max(a,u)$. A maximum matching pairs every node of the smaller side, so the nodes of the smaller side choose different partners one after another: K ways for the first, $K-1$ for the second, and so on down to $K-k+1$. This gives

$$K(K-1)\cdots(K-k+1) = K! \,/\, (K-k)!$$

matchings. The answer is the product of this value for the A–U group and for the C–G group. Counting the letters takes $O(n)$ time and Python integers have no size limit, so large factorials need no special handling.

4.3 Python code

```
from math import factorial

SAMPLE_RNA = "AUGCUUC"

def pair_count(x, y):
    """Ways to form the maximum number of pairs between two groups: max! / (max - min)!"""
    big, small = max(x, y), min(x, y)
    return factorial(big) // factorial(big - small)

def run_case(rna, name):
    rna = "".join(rna.split()).upper()
    print(f"\n===== {name} =====")
    invalid = sorted(set(rna) - set("AUGC"))
    if not rna or invalid:
```

```
        print("Invalid input:", "empty sequence" if not rna else "bad symbol(s) " + ", ".join(invalid))
        return
    c = {b: rna.count(b) for b in "AUGC"}
    print(f"RNA sequence : {rna}  (length {len(rna)})")
    print("Base counts  : " + "  ".join(f"{b}={c[b]}" for b in "AUGC"))

    total = 1
    for x, y in ("AU", "CG"):
        big, small = max(c[x], c[y]), min(c[x], c[y])
        ways = pair_count(c[x], c[y])
        print(f"{x}-{y}: {small} pairs, {big}!/{big - small}! = {ways}")
        total *= ways
    print(f"\nMaximum matchings (A-U x C-G) = {total}")
    print(total)                            # Rosalind output

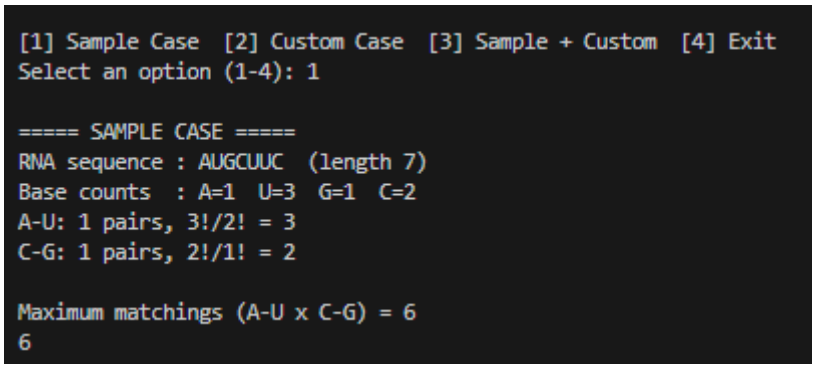
def main():
    while True:
        print("\n[1] Sample Case  [2] Custom Case  [3] Sample + Custom  [4] Exit")
        choice = input("Select an option (1-4): ").strip()
        if choice in ("1", "3"):
            run_case(SAMPLE_RNA, "SAMPLE CASE")
        if choice in ("2", "3"):
            run_case(input("Enter RNA sequence (A, U, G, C): "), "CUSTOM CASE")
        if choice == "4":
            break
        if choice not in ("1", "2", "3"):
            print("Invalid option.")

if __name__ == "__main__":
    try:
        main()
    except (EOFError, KeyboardInterrupt):
        print("\nProgram terminated.")
```

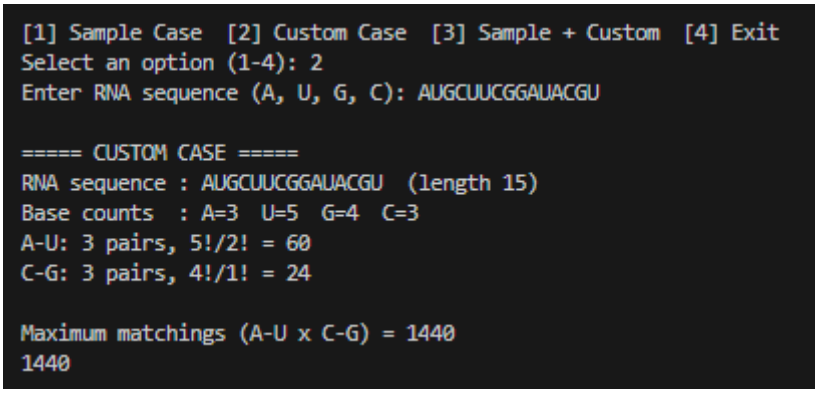
4.4 Explanation of the implementation

1. **Formula.** `pair_count(x, y)` returns `max(x,y)! // (max(x,y) - min(x,y))!`, the number of maximum matchings between two groups of sizes `x` and `y`.
2. **Input check.** `run_case()` removes spaces, converts the text to upper case and prints an error if the string is empty or contains a letter other than A, U, G, C.
3. **Counting.** The number of each base is found with `str.count`. For the A–U group and then the C–G group the program prints the number of pairs and the factorial ratio, and multiplies the two results in `total`.
4. **Output.** The final answer is printed on its own line, as Rosalind expects.
5. **Menu.** `main()` repeats the menu (sample, custom, sample plus custom, exit) until the user chooses 4, and `try/except` ends the program cleanly on Ctrl+C or end of input.

4.5 Output snapshot



MMCH.py, sample case.



MMCH.py, custom case.

5 Conclusion

Three Rosalind problems were solved in Python and checked against their sample outputs: LOCA returns 23 (LYPRTEINSTRIN and LYEINSTEIN is one of the two optimal alignments listed), GAFF returns 8 with PRT---EINS and PRTWPSEIN-, and MMCH returns 6. LOCA and GAFF show how dynamic-programming tables replace a search over exponentially many alignments with $O(nm)$ work; the three states M, X and Y of GAFF show how an affine gap penalty needs extra information about whether a gap is open. MMCH shows that once the independent parts of a counting problem are identified, a short formula gives the answer directly.